

Unit 3

Parallel Processing & Pipelining

- ☐ Parallel Processing
- ☐ Pipelining
- ☐ Instruction Pipeline
- ☐ Pipeline Hazards

Curated By: Dr. Rohit Mathur



- **Computer Architecture: (Ref: M. M. Mano)**
 - Introduction, computer types, functional units, Von Neumann and Harvard architectures, RISC and CISC Architectures.
 - Register Transfer and Microoperations, Design of Basic Computer, Design of Accumulator Logic, Control unit design, **Parallel Processing, Pipelining**, Input-Output Organization, Memory Organization.
- **8086 Microprocessor:**
 - 8086 Architecture, 8086 Instruction Set: types of instructions and addressing modes, assembler and assembler directives, assembly language programming.
- **RISC V:**
 - Introduction to the Reduced Instruction Set Computer, RISC V architecture and features.

- 1) M. Morris Mano, “Computer System Architecture”, Pearson, (3e), 2007.**
- 2) K. M . Bhurchandi, A K Ray, Advanced Microprocessors and Peripheral Devices”, McGraw Hill, (3e), 2018.**
- 3) David A. Patterson, and John L. Hennessy, Computer Organization and Design, The Hardware / Software Interface: RISC-V Edition, Morgan Kaufmann, 2018.**
- 4) Sonal Yadav, Computer System Organization, 1 (e), All India Council for Technical Education, 2024.**
- 5) V.C. Hamacher, Z. Vranesic & S. Zaky, “Computer Organization”, McGraw Hill, (5e), 2002.**

- By the end of this course, students will be able to:
- 1) Demonstrate understanding of Register Transfer and Microoperations.
 - 2) Demonstrate understanding of control unit and its subsystem design.
 - 3) Explain concepts and impact of cache memory, pipelining and parallel processing on system performance.
 - 4) Explain the operation of various addressing modes of 8086 microprocessor.
 - 5) Develop Logical and Conditional assembly language programming skills for 8086 microprocessor.

❑ Throughput

- ❑ The amount of processing that can be accomplished during a given interval of time.
- ❑ It is a vital indicator of overall processing capacity, often measured in instructions per second, transactions per second, or bits transferred per second.

❑ Instruction Throughput:

- ❑ Often described as $\text{Instructions} \div \text{Second}$, or defined by $\text{Cycles} \div \text{Instruction (CPI)}$. High throughput means minimizing the time a CPU spends on overhead, stalls, or waiting for memory.

❑ Data / Memory Throughput:

- ❑ Expressed as gigabytes or megabytes per second (GB/s, MB/s). It measures how fast data can be read from or written to storage, or moved between cache levels and memory.

❑ How Architects Maximize Throughput?

❑ Computer architects utilize several techniques to increase a system's throughput, often at the expense of slight increases in individual task latency:

❑ **Pipelining:** Breaking instructions into smaller, sequential steps so multiple instructions can overlap in execution.

❑ **Parallelism:** Adding multiple execution units or cores to process different tasks concurrently.

❑ **Caching:** Storing frequently accessed data in smaller, ultra-fast memory near the CPU to prevent time-consuming trips to main memory.

❑ **Vector Processing:** Processing large datasets in single operations (SIMD - Single Instruction, Multiple Data) instead of iterating through them one by one.

❑ PARALLEL PROCESSING:

- ❑ Execution of Concurrent Events in the computing process to achieve faster computational Speed.
- ❑ Parallel processing computers are required to meet the demands of large scale computations in many scientific, engineering, military, medical, artificial intelligence, and basic research areas.
- ❑ The following are some representative applications of parallel processing computers:
Numerical weather forecasting, computational aerodynamics, finite-element analysis, remote-sensing applications, genetic engineering, computer-assisted tomography, and weapon research and defence.

Execution of *Concurrent Events* in the computing process to achieve faster *Computational Speed*

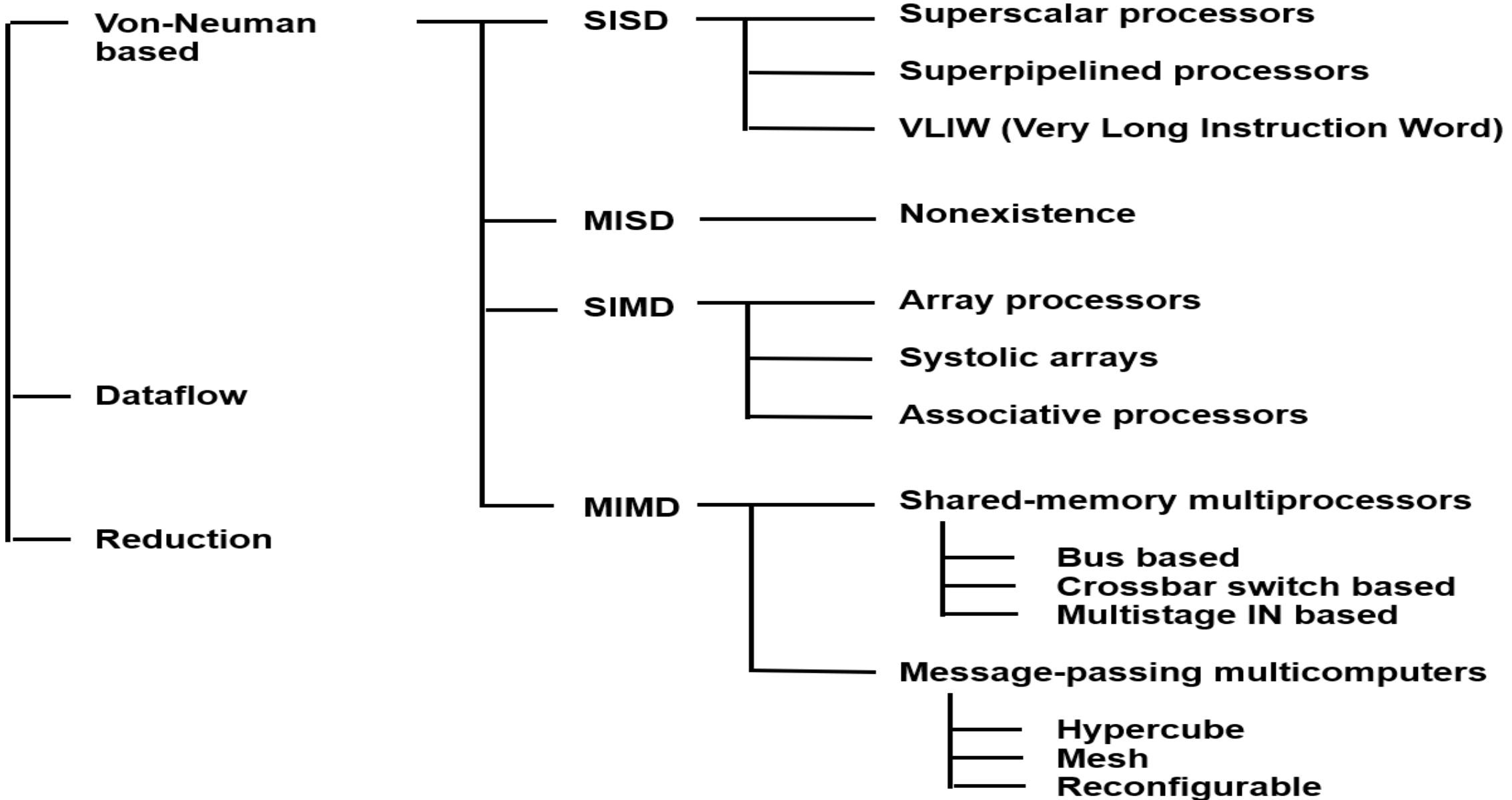
Levels of Parallel Processing

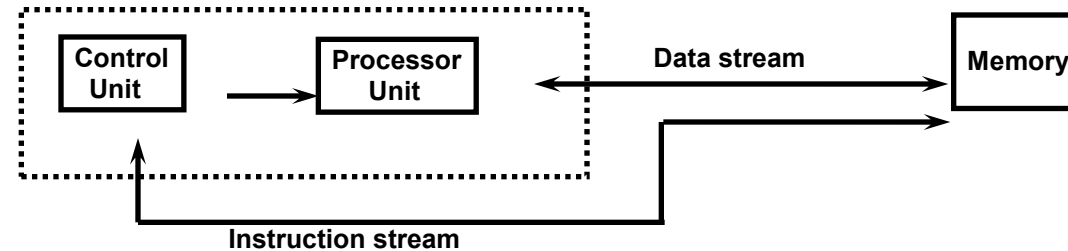
- Job or Program level
- Task or Procedure level
- Inter-Instruction level
- Intra-Instruction level

Architectural Classification

- Flynn's classification
 - Based on the multiplicity of *Instruction Streams* and *Data Streams*
 - Instruction Stream
 - Sequence of Instructions read from memory
 - Data Stream
 - Operations performed on the data in the processor

		Number of <i>Data Streams</i>	
		Single	Multiple
Number of <i>Instruction Streams</i>	Single	SISD	SIMD
	Multiple	MISD	MIMD





Characteristics

- Standard von Neumann machine
- Instructions and data are stored in memory
- One operation at a time

Limitations

Von Neumann bottleneck

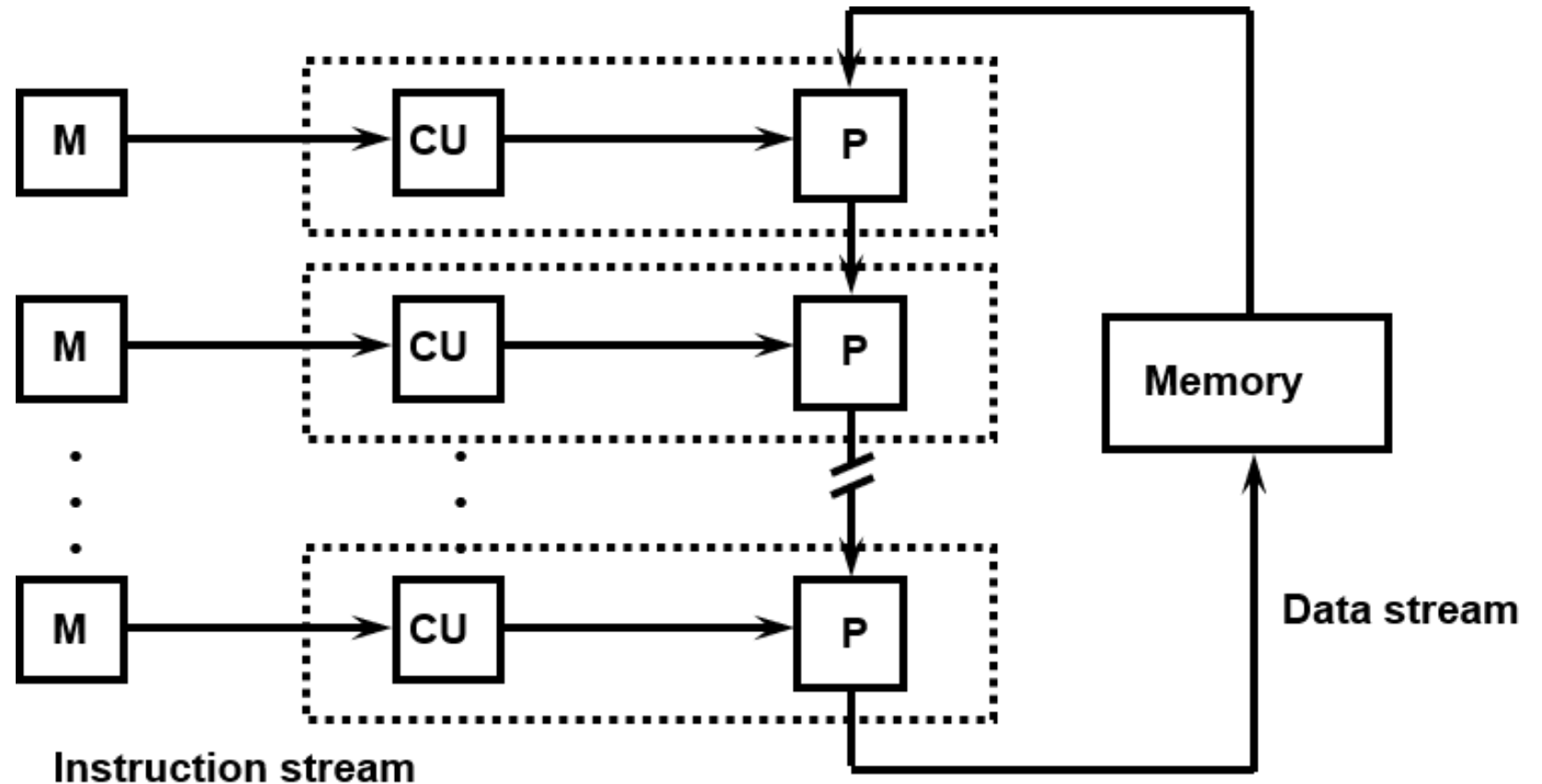
Maximum speed of the system is limited by the *Memory Bandwidth* (bits/sec or bytes/sec)

- Limitation on *Memory Bandwidth*
- Memory is shared by CPU and I/O

- **Multiprogramming**
- **Spooling**
- **Multifunction processor**
- **Pipelining**
- **Exploiting instruction-level parallelism**
 - **Superscalar**
 - **Superpipelining**
 - **VLIW (Very Long Instruction Word)**

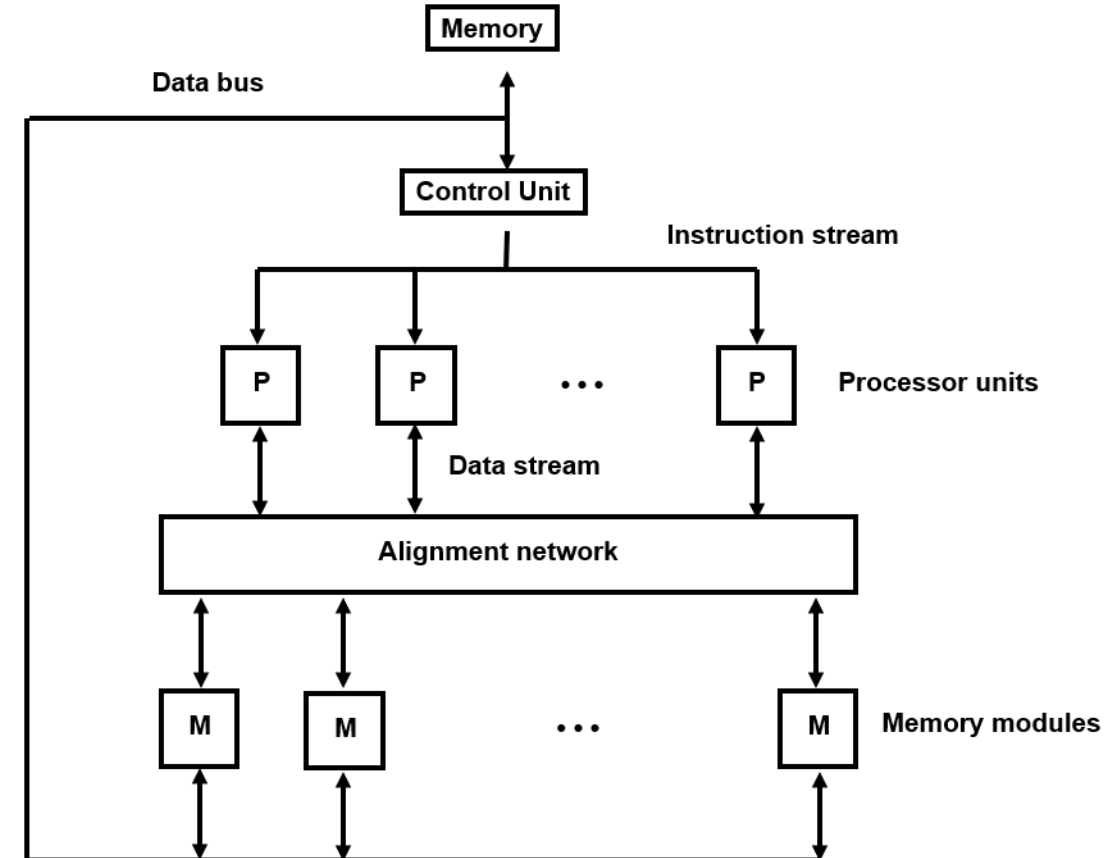
Characteristics

- There is no computer at present that can be classified as MISD



Characteristics

- Only one copy of the program exists
- A single controller executes one instruction at a time



Array Processors

- The control unit broadcasts instructions to all PEs, and all active PEs execute the same instructions
- ILLIAC IV, GF-11, Connection Machine, DAP, MPP

Systolic Arrays

- Regular arrangement of a large number of very simple processors constructed on VLSI circuits
- CMU Warp, Purdue CHiP

Associative Processors

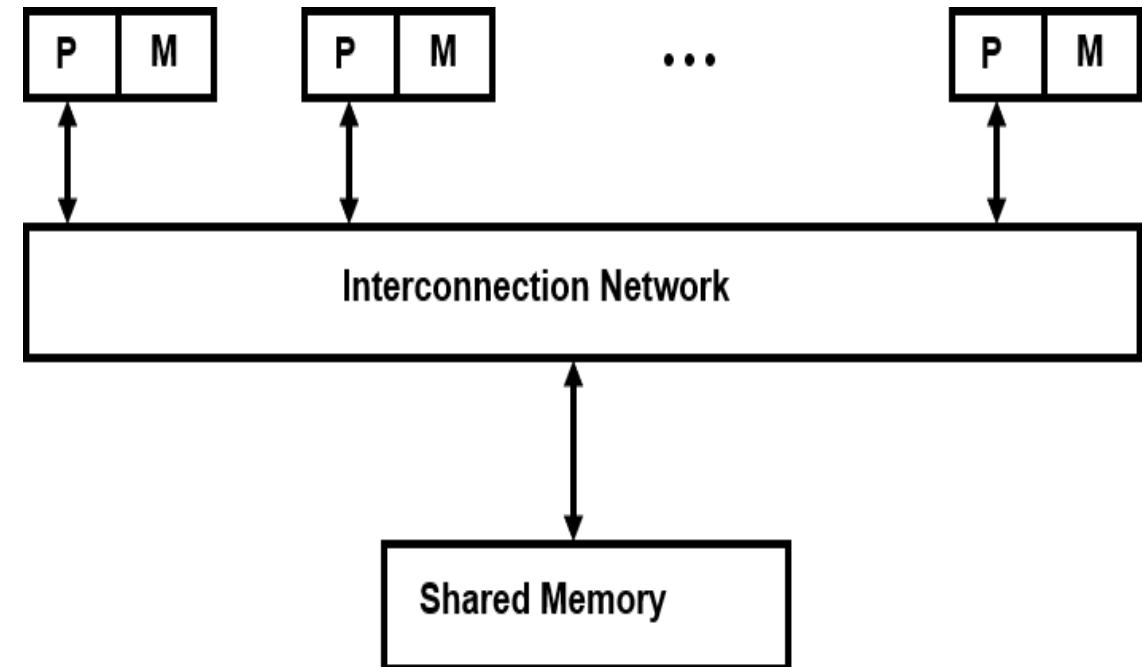
- Content addressing
- Data transformation operations over many sets of arguments with a single instruction
- STARAN, PEPE

Characteristics

- Multiple processing units
- Execution of multiple instructions on multiple data

Types of MIMD computer systems

- Shared memory multiprocessors
- Message-passing multicomputers



Characteristics

All processors have equally direct access to one large memory address space

Example systems

Bus and cache-based systems

- Sequent Balance, Encore Multimax

Multistage IN-based systems

- Ultracomputer, Butterfly, RP3, HEP

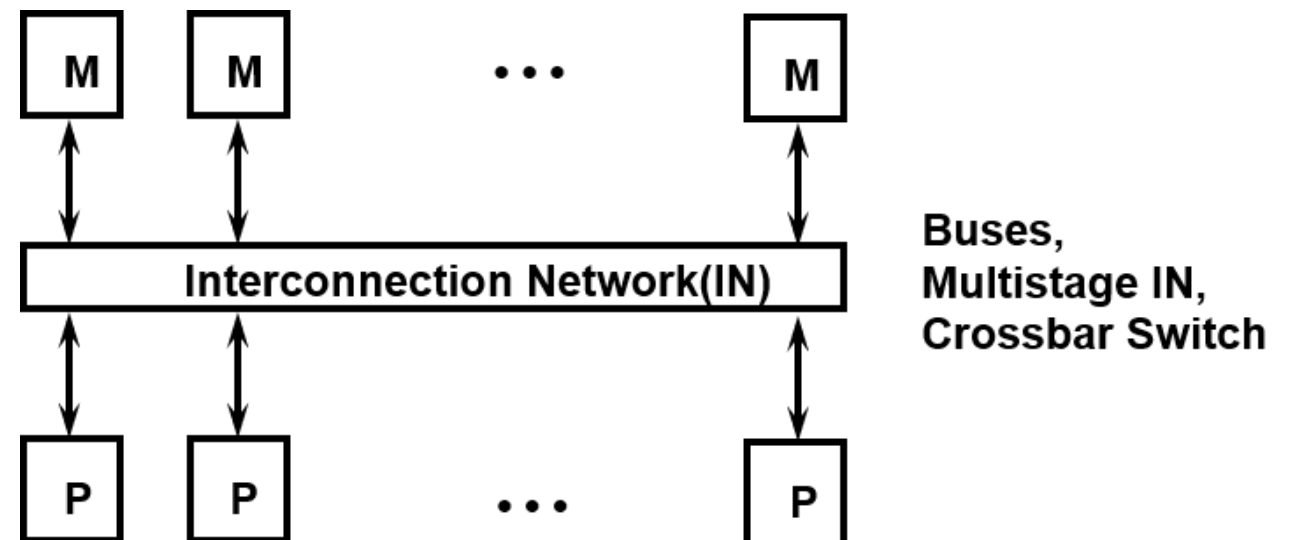
Crossbar switch-based systems

- C.mmp, Alliant FX/8

Limitations

Memory access latency

Hot spot problem



Characteristics

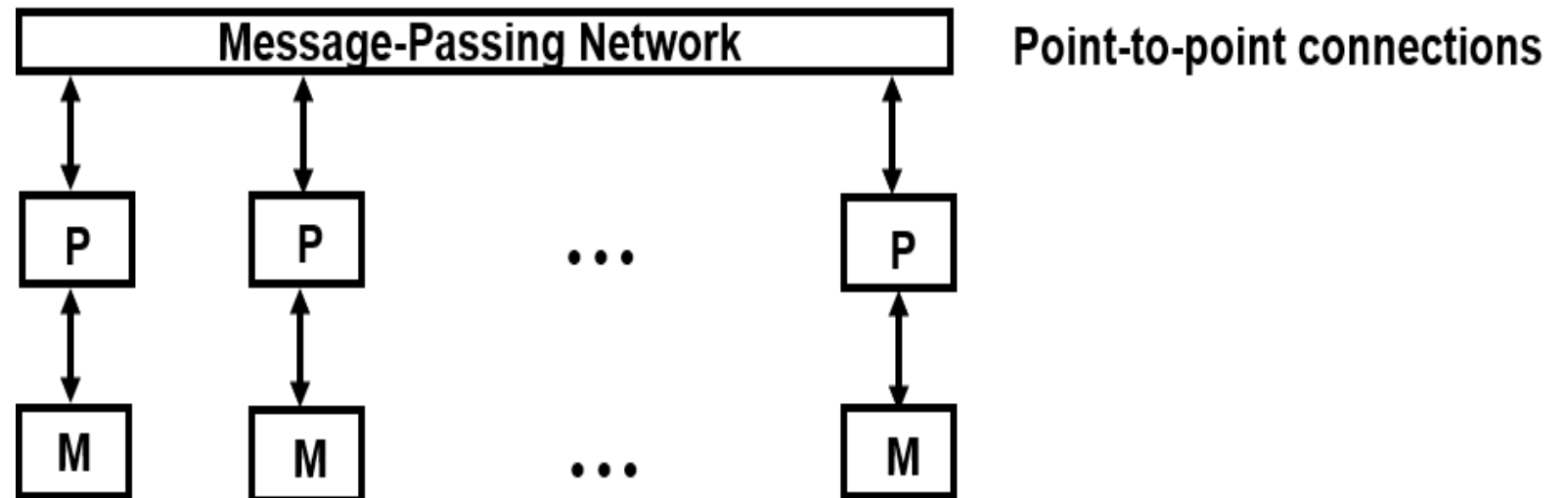
- Interconnected computers
- Each processor has its own memory, and communicate via message-passing

Example systems

- Tree structure: Teradata, DADO
- Mesh-connected: Rediflow, Series 2010, J-Machine
- Hypercube: Cosmic Cube, iPSC, NCUBE, FPS T Series, Mark III

Limitations

- Communication overhead
- Hard to programming

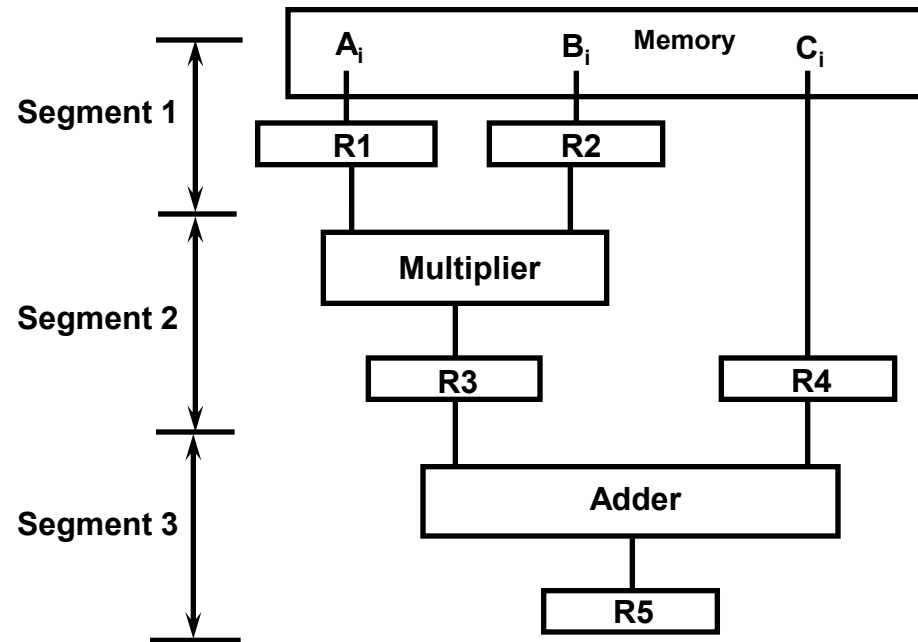


❑ Pipeline processing:

- ❑ It is an implementation technique where arithmetic suboperations or the phases of a computer instruction cycle overlap in execution.
- ❑ Pipelining is a technique of decomposing a sequential process into suboperations, with each subprocess being executed in a special dedicated segment that operates concurrently with all other segments.
- ❑ A pipeline can be visualized as a collection of processing segments through which binary information flows.
- ❑ The name “pipeline” implies a flow of information analogous to an industrial assembly line.
- ❑ It is characteristic of pipelines that several computations can be in progress in distinct segments at the same time.
- ❑ The overlapping of computation is made possible by associating a register with each segment in the pipeline.
- ❑ The registers provide isolation between each segment so that each can operate on distinct data simultaneously.

A technique of decomposing a sequential process into suboperations, with each subprocess being executed in a partial dedicated segment that operates concurrently with all other segments.

$$A_i * B_i + C_i \quad \text{for } i = 1, 2, 3, \dots, 7$$



$R1 \leftarrow A_i, R2 \leftarrow B_i$	Load A_i and B_i
$R3 \leftarrow R1 * R2, R4 \leftarrow C_i$	Multiply and load C_i
$R5 \leftarrow R3 + R4$	Add

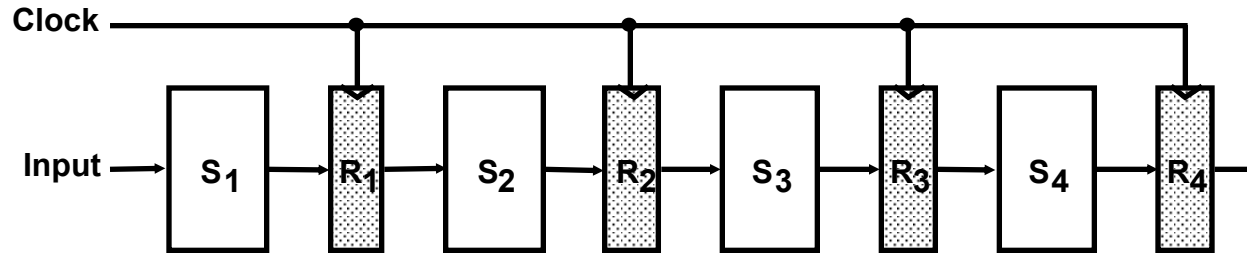


OPERATIONS IN EACH PIPELINE STAGE



Clock Pulse	Segment 1		Segment 2		Segment 3
Number	R1	R2	R3	R4	R5
1	A1	B1			
2	A2	B2	$A1 * B1$	C1	
3	A3	B3	$A2 * B2$	C2	$A1 * B1 + C1$
4	A4	B4	$A3 * B3$	C3	$A2 * B2 + C2$
5	A5	B5	$A4 * B4$	C4	$A3 * B3 + C3$
6	A6	B6	$A5 * B5$	C5	$A4 * B4 + C4$
7	A7	B7	$A6 * B6$	C6	$A5 * B5 + C5$
8			$A7 * B7$	C7	$A6 * B6 + C6$
9					$A7 * B7 + C7$

General Structure of a 4-Segment Pipeline



Space-Time Diagram

		1	2	3	4	5	6	7	8	9	Clock cycles
Segment	1	T1	T2	T3	T4	T5	T6				
	2		T1	T2	T3	T4	T5	T6			
	3			T1	T2	T3	T4	T5	T6		
	4				T1	T2	T3	T4	T5	T6	

n: Number of tasks to be performed

Conventional Machine (Non-Pipelined)

t_n : Clock cycle

τ_1 : Time required to complete the n tasks

$$\tau_1 = n * t_n$$

Pipelined Machine (k stages)

t_p : Clock cycle (time to complete each suboperation)

τ_k : Time required to complete the n tasks

$$\tau_k = (k + n - 1) * t_p$$

Speedup

S_k : Speedup

$$S_k = n * t_n / (k + n - 1) * t_p$$

$$\lim_{n \rightarrow \infty} S_k = \frac{t_n}{t_p} \quad (= k, \text{ if } t_n = k * t_p)$$

Example

- 4-stage pipeline
- suboperation in each stage; $t_p = 20\text{nS}$
- 100 tasks to be executed
- 1 task in non-pipelined system; $20 \times 4 = 80\text{nS}$

Pipelined System

$$(k + n - 1) \times t_p = (4 + 99) \times 20 = 2060\text{nS}$$

Non-Pipelined System

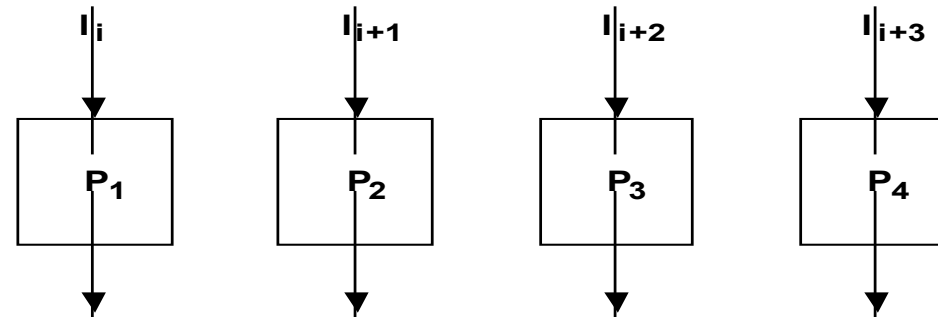
$$n \times k \times t_p = 100 \times 80 = 8000\text{nS}$$

Speedup

$$S_k = 8000 / 2060 = 3.88$$

4-Stage Pipeline is basically identical to the system with 4 identical function units

Multiple Functional Units



- ❑ There are two areas of computer design where the pipeline organization is applicable.
- ❑ An **arithmetic pipeline** divides an arithmetic operation into sub operations for execution in the pipeline segments.
- ❑ An **instruction pipeline** operates on a stream of instructions by overlapping the fetch, decode, and execute phases of the instruction cycle.
- ❑ The two types of pipelines are explained in the following sections.

❖ SIX PHASES IN INSTRUCTION CYCLE

- [1] Fetch an instruction from memory
- [2] Decode the instruction
- [3] Calculate the effective address of the operand
- [4] Fetch the operands from memory
- [5] Execute the operation
- [6] Store the result in the proper place

- * Some instructions skip some phases
- * Effective address calculation can be done in the part of the decoding phase
- * Storage of the operation result into a register is done automatically in the execution phase

❖ 4-Stage Pipeline

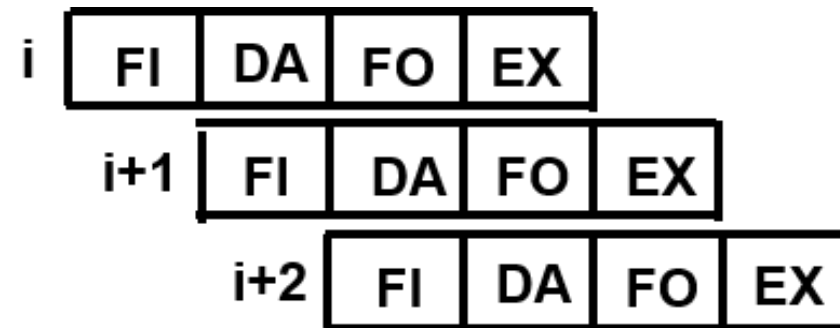
- [1] FI: Fetch an instruction from memory
- [2] DA: Decode the instruction and calculate the effective address of the operand
- [3] FO: Fetch the operand
- [4] EX: Execute the operation

❖ Execution of Three Instructions in a 4-Stage Pipeline

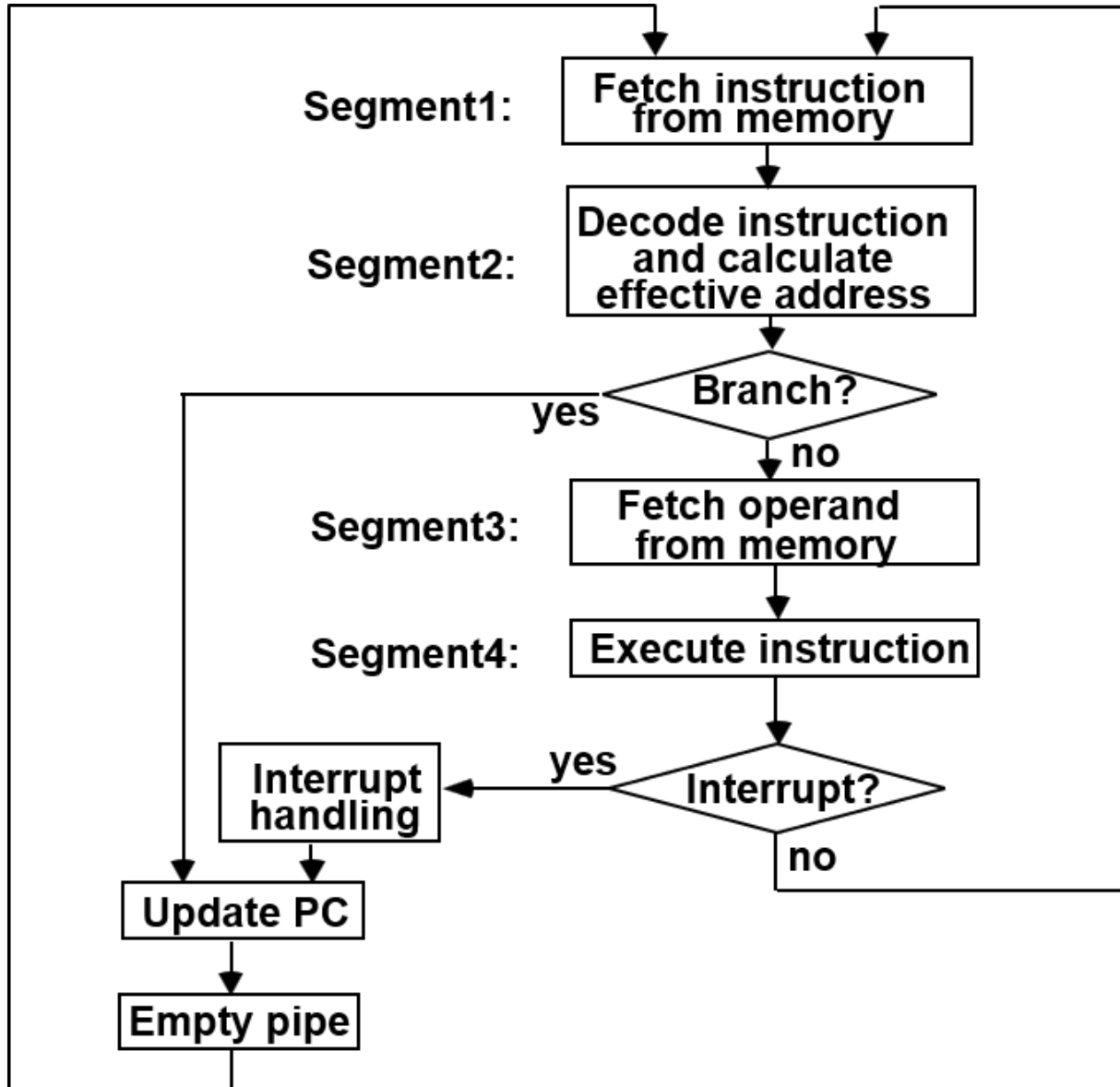
❖ Conventional



❖ Pipelined



INSTRUCTION EXECUTION IN A 4-STAGE PIPELINE



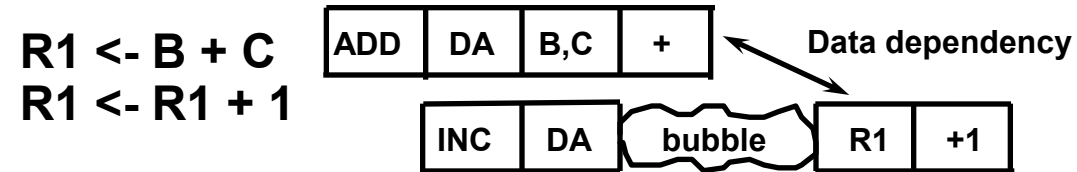
Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction 1	FI	DA	FO	EX									
Instruction 2		FI	DA	FO	EX								
(Branch) Instruction 3			FI	DA	FO	EX							
Instruction 4				FI	-	-	FI	DA	FO	EX			
Instruction 5					-	-	-	FI	DA	FO	EX		
Instruction 6									FI	DA	FO	EX	
Instruction 7										FI	DA	FO	EX

Structural hazards(Resource Conflicts)

Hardware Resources required by the instructions in simultaneous overlapped execution cannot be met

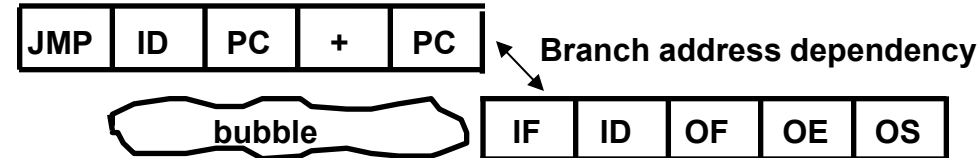
Data hazards (Data Dependency Conflicts)

An instruction scheduled to be executed in the pipeline requires the result of a previous instruction, which is not yet available



Control hazards

Branches and other instructions that change the PC make the fetch of the next instruction to be delayed



Hazards in pipelines may make it necessary to **stall** the pipeline

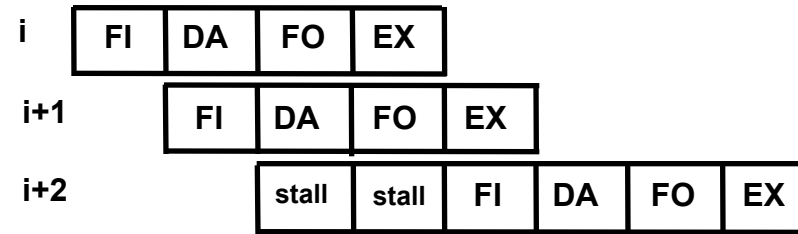


Pipeline Interlock:
Detect Hazards Stall until it is cleared

Structural Hazards

Occur when some resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute

Example: With one memory-port, a data and an instruction fetch cannot be initiated in the same clock



The Pipeline is stalled for a structural hazard
<- Two Loads with one port memory
-> Two-port memory will serve without stall

Data Hazards

Occurs when the execution of an instruction depends on the results of a previous instruction

```
ADD    R1, R2, R3
SUB    R4, R1, R5
```

Data hazard can be dealt with either hardware techniques or software technique

Hardware Technique

Interlock

- hardware detects the data dependencies and delays the scheduling of the dependent instruction by stalling enough clock cycles

Forwarding (bypassing, short-circuiting)

- Accomplished by a data path that routes a value from a source (usually an ALU) to a user, bypassing a designated register. This allows the value to be produced to be used at an earlier stage in the pipeline than would otherwise be possible

Software Technique

Instruction Scheduling(compiler) for *delayed load*

Example:

ADD R1, R2, R3

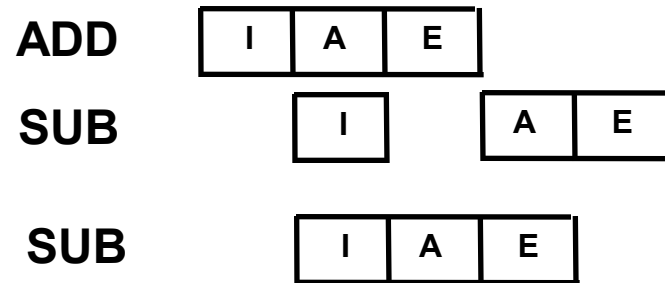
SUB R4, R1, R5

3-stage Pipeline

I: Instruction Fetch

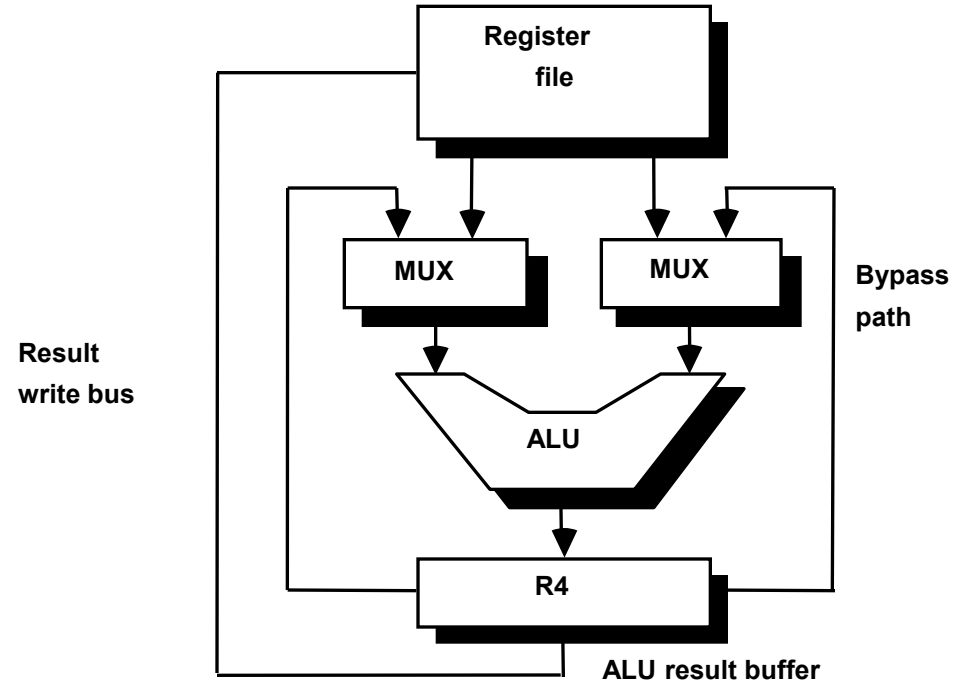
A: Decode, Read Registers, ALU Operations

E: Write the result to the destination register



Without Bypassing

With Bypassing



**a = b + c;
d = e - f;**

Unscheduled code:

```
LW    Rb, b
LW    Rc, c
→ ADD Ra, Rb, Rc
→ SW  a, Ra
LW    Re, e
LW    Rf, f
→ SUB Rd, Re, Rf
→ SW  d, Rd
```

Scheduled Code:

```
LW    Rb, b
LW    Rc, c
LW    Re, e
ADD   Ra, Rb, Rc
LW    Rf, f
SW    a, Ra
→ SUB Rd, Re, Rf
SW    d, Rd
```

Delayed Load

A load requiring that the following instruction not use its result

- * Prefetch Target Instruction
- * Branch Target Buffer
- * Loop Buffer
- * Branch Prediction
- * Delayed Branch

Prefetch Target Instruction

- Fetch instructions in both streams, branch not taken and branch taken
- Both are saved until branch branch is executed. Then, select the right instruction stream and discard the wrong stream

Branch Target Buffer(BTB; Associative Memory)

- Entry: Addr of previously executed branches; Target instruction and the next few instructions
- When fetching an instruction, search BTB.
- If found, fetch the instruction stream in BTB;
- If not, new stream is fetched and update BTB

Loop Buffer(High Speed Register file)

- Storage of entire loop that allows to execute a loop without accessing memory

Branch Prediction

- Guessing the branch condition, and fetch an instruction stream based on the guess.
- Correct guess eliminates the branch penalty

Delayed Branch

- Compiler detects the branch and rearranges the instruction sequence by inserting useful instructions that keep the pipeline busy in the presence of a branch instruction



Thankyou!